Financial Literacy Quiz System

A practical, build-it-yourself guide — stack, QR code, shuffle logic, leaderboard, admin page, and free-hosting feasibility

1. What You're Building — In One Page

One website. Participant opens it (via QR code scan), sees a fun entry screen (not the old wheel), gets a difficulty tier, then gets 20 questions out of a 1000-question bank — shuffled so each person gets a different set and order. Answers are timed. When 500 people are doing this at once, a live leaderboard ranks everyone by correct answers (fastest as tie-breaker). An admin page shows you exactly who's winning, live, and lets you export the final results.

- Frontend: the website participants use on their phone (entry screen + quiz + their live rank).
- Backend: the server logic — picks/shuffles questions, checks answers, calculates scores.
- Database: stores the 1000 questions, every participant, and every answer.
- Admin Panel: a separate password-protected page just for you/organizers.

2. Tech Stack (What to Actually Use)

Part	Use This	Why
Backend	Python + FastAPI	Fast to write, you already know Python, handles many users at once well (async)
Database	PostgreSQL (or SQLite for a quick test)	Stores questions/participants/answers reliably
Fast leaderboard	Redis (optional but recommended)	Keeps live rankings fast even with 500 people hitting it at once
Frontend	React (or even plain HTML/JS if you want it simpler)	Builds the entry screen + quiz + leaderboard views
Realtime updates	WebSockets, or just refresh every 3 sec	So leaderboard updates live without a full page reload
Hosting	Render / Railway / Fly.io (all have free tiers)	No server management, deploy straight from GitHub

3. How the QR Code Works

The QR code is not magic — it's just a picture that encodes your website's URL. Once you deploy your site (say, https://nse-quiz.onrender.com), you generate a QR code image that points to that link. Scanning it with any phone camera just opens that URL in the browser.

Easiest way (no design needed):

- Use a free QR generator like qr-code-generator.com or the built-in QR feature in Google Chrome (share icon → QR code) — paste your live URL, download the PNG, print it.

Or generate it yourself in Python (useful if you want to auto-generate one per event/session):

```
pip install qrcode[pil]

import qrcode
img = qrcode.make("https://nse-quiz.onrender.com/session/aug2026")
img.save("quiz_qr.png")
```

Tip: put the session ID in the URL itself (like above) — that way one QR code always points to the currently active event, and you don't need to reprint it for every event if you just reuse the same session slug.

4. The Entry Interface (Instead of the Wheel)

You want something interactive but not the spinning wheel. Simplest good option to actually build: a tap-to-flip card or a scratch card. Both need just 3 lines of underlying logic — the animation is the only 'hard' part, and even that's a well-known CSS trick.

- Tap-to-Flip Card: show a card, participant taps it, it flips (pure CSS 3D flip transform) revealing Low/Medium/High.
- Scratch Card: use a element with a grey overlay; participant swipes with their finger, canvas erases to reveal the tier underneath (a small JS library like 'scratchcard-js' does this in ~20 lines).
- Either way: the tier shown must be confirmed by your backend (send a request, backend randomly assigns Low/Medium/High and returns it) — don't decide the tier only in the browser, or people could inspect the page and pick 'Low' every time.

Backend logic for tier assignment (FastAPI example):

```
import random

@app.post("/participants/{pid}/tier")
def assign_tier(pid: str):
    tier = random.choice(["low", "medium", "high"])
    save_tier_to_db(pid, tier)
    return {"tier": tier}
```

5. Shuffling Questions So Everyone Gets Different Ones

This is the core trick and it's simple: store all questions in the database tagged by tier (low/medium/high). When a participant starts, randomly pick 20 questions from that tier's pool, then shuffle both the question order and each question's 4 answer options.

```
import random

def get_quiz_for_participant(tier: str):
    pool = get_questions_by_tier(tier)      # e.g. ~333 questions per tier
    selected = random.sample(pool, 20)      # 20 random, no repeats
    random.shuffle(selected)                # shuffle question order
    for q in selected:
        random.shuffle(q.options)           # shuffle answer options too
    return selected
```

random.sample() never repeats a question within one draw, and because it's random per participant, two people sitting next to each other will almost never see the same 20 questions in the same order — exactly like the old system.

6. Live Numbering / Leaderboard

Every time a participant finishes, calculate their score and update a live ranking. Simplest reliable approach for 500 concurrent users: use Redis's built-in sorted-set type — it's built exactly for leaderboards and stays fast no matter how many people finish at once.

```
# On finish:
score = (correct_count * 100) - (time_taken_seconds * 0.5)
redis.zadd("leaderboard", {participant_id: score})

# To show top 10, live:
top10 = redis.zrevrange("leaderboard", 0, 9, withscores=True)

# To show one participant's own rank:
rank = redis.zrevrank("leaderboard", participant_id)
```

If you'd rather skip Redis to keep things simpler for a first version, you can do the same with a plain SQL query (ORDER BY score DESC LIMIT 10) — it'll be a bit slower under heavy concurrent load, but for ~500 users it's usually still fine, especially if you refresh the leaderboard every 2-3 seconds instead of on every single answer.

On the participant's own screen after they finish, just show: 'You scored 18/20 — Rank #23 of 214 so far.' That number updates as more people complete the quiz, using the query above.

7. Admin Page — Who's Winning

A second, separate part of the website — behind a simple password/login — just for you. Doesn't need to be fancy.

- Login screen (a single hardcoded admin password is fine for an internal event tool).
- A live table: rank, name, score, correct answers, time taken — pulled from the same leaderboard data, refreshing every few seconds.
- A big 'Winner' card at the top showing rank #1 clearly.
- A button to export everything (all participants + all answers) as a CSV/Excel file once the event ends — so you have a record without needing to touch the database directly.
- A button to start/stop the session (so the quiz link only accepts entries during the actual event window).

Minimal export logic (Python):

```
import csv

@app.get("/admin/export")
def export_results():
    rows = get_all_participants_with_scores()
    with open("results.csv", "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["Name", "Tier", "Correct", "Time (s)", "Score", "Rank"])
        for r in rows:
            writer.writerow([r.name, r.tier, r.correct, r.time, r.score, r.rank])
    return FileResponse("results.csv")
```

8. Can This Be Built for Free? — Yes, With Limits

You can build and run this entire system at zero cost for a moderate-size event. Here's exactly where the free limits are, so you know when you'd actually need to pay:

Piece	Free Option	Where the Free Limit Bites
Backend hosting	Render.com or Railway.app free tier	Free tier servers 'sleep' when idle and can be slow to wake up, and have limited RAM/CPU — fine for testing, risky for exactly-500-at-once on event day
Database	Free Postgres on Render/Railway/Supabase (usually ~500MB–1GB free)	1000 questions + 500 participants + 10,000 answer rows is tiny — well within free limits
Redis	Free tier on Upstash or Redis Cloud (~ small free quota)	More than enough for a leaderboard of 500 people
Frontend hosting	Vercel / Netlify free tier	Very generous free limits — practically a non-issue here
QR code	Free online generator or the Python script above	Completely free, no limit
Domain name	Optional — free hosting URLs work fine (e.g. yourapp.onrender.com)	Only costs money if you specifically want a custom domain like nsequiz.in

Realistic recommendation: build and test everything on free tiers first. If your actual event needs guaranteed performance for 500 simultaneous users (no risk of a slow 'waking up' server at the worst moment), upgrade just the backend hosting to a small paid tier for that one day (e.g. Render's cheapest paid plan) — usually a few hundred to a low thousand rupees for the month, not an ongoing cost if you shut it down after. Everything else (DB, Redis, frontend, QR code) can comfortably stay free even on event day.

9. Suggested Build Order

1. Set up FastAPI + Postgres locally, create the Question/Participant tables.
2. Import your 1000 questions (write a small script to bulk-insert from a CSV).
3. Build the shuffle-and-serve-20-questions endpoint, test it manually.
4. Build the quiz-taking frontend (question by question, submit answers, timer).
5. Add the scoring + leaderboard (start with plain SQL ranking, add Redis later if needed).
6. Build the entry interface (tap-to-flip card) and hook it to the tier-assignment endpoint.
7. Build the admin page (login, live table, winner card, CSV export).
8. Deploy everything on free tiers, generate the QR code, test end-to-end with a few friends.
9. Load-test with a script simulating 500 fake users hitting it at once before the real event.